

Reply to: J. Daniel Garcia (josedaniel.garcia@uc3m.es)

Support for contract based programming in C++

G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup

1. Introduction

This paper is based on P0380 ([P0380R1 \[1\]](#)). All changes are relative to latest working draft ([N4700 \[2\]](#)).

2. Summary of wording

2.1. A new attributes syntax

To simplify the way that contracts are specified, and in line with our previous design paper, we propose a new syntax that may be used for attributes.

We propose this new syntax to be usable by attributes taking a single argument, which is a valid conditional expression.

```
[[contract-attribute: conditional-expression]]
```

where a *contract-attribute* is one of the following: **expects**, **ensures**, **assert**.

To avoid confusion, only the colon syntax for attributes is permitted in contract specifications

We also needed to support the idea of assertion levels needed by contracts. For this reason, we have introduced the concept of an attribute modifier that may appear after the attribute token itself.

```
[[contract-attribute modifier: conditional-expression]]
```

In this way we may specify contracts with assertion levels easily. We require that each attribute using the new proposed attribute syntax explicitly lists its accepted modifiers (if any).

In the case of contract attributes valid modifiers are: **always**, **axiom**, **default**, and **audit**

Finally, we needed to introduce the ability to define the return value to be used in postconditions. We allow, that an **expects** attribute lists an identifier which is associated with the return value of a function.

```
[[expects modifier identifier: conditional-expression]]
```

With all this changes we can easily specify a contract:

```
int f(int x)
[[expects audit: x>0]]
[[ensures axiom res: res>1]];

void g() {
    int x = f(5);
    int y = f(12);
    //...
[[assert: x+y>0]]
}
```

Note that while **assert(expression)** would expand as a function-like macro with the appropriate header, **assert:** is not a function-

like invocation, so does not expand.

2.2. Functions versus function types

The current standard establishes a distinction between an attribute applied to a function and to the function type (the normative text uses the form "*appertain to function type*"). With that approach there would be a difference between the following cases:

```
[[attribute]] void f(); // Attribute applied to function
void f [[attribute]] (); // Attribute applied to function
void f() [[attribute]]; // Attribute applied to function type
```

Only the third option is interesting for contract attributes as the preconditions and postconditions need make use of function's parameters.

```
void f(int x, int y)
  [[expects: x>0]]
  [[expects: y!=0]]
  [[ensures result: result > x+y]];
```

Consequently contracts attributes (as any other attribute in that syntactical location) *appertain to the function type*. However, they are not part of the function type.

Note that this does not solve the issue of being able to use attributes on lambda expressions (see Core issue 2097). In fact, until that issue is not solved it will not be possible to specify preconditions and postconditions for lambda expressions.

```
void f() {
  // Not currently supported
  auto increment = [](int x) [[expects: x>0]] { return x+1; };
  // ...
}
```

2.3 Contracts repetition

We require that any redeclaration of a function either has the contract or completely omits it.

```
int f(int x)
  [[expects: x>0]]
  [[ensures r: r>0]];

int f(int x); // OK. No contract.

int f(int x)
  [[expects: x>=0]]; // Error missing ensures and different expects condition

int f(int x)
  [[expects: x>0]]
  [[ensures r: r>0]]; //OK. Same contract.
```

Different argument names in redeclaration would be usually considered irrelevant.

```
int f(int x)
  [[expects: x>0]]
  [[ensures r: r>0]];

int f(int y)
  [[expects: y>0]] // Should be OK
  [[ensures z: z>0]]; // Should be OK
```

Consequently, we require that contracts are the same in redeclarations. That means that:

- Preconditions and postconditions appear in the same order.
- Their modifiers are the same.
- Each conditional expression would satisfy the ODR if it appeared in function definition, except for renaming of parameters and return value identifiers.

2.4 Structured bindings and postconditions

One might imagine using structured bindings in postconditions:

```
std::tuple f()  
[[ensures [x,y]: x>0 && y.size()>0]];
```

However, we decided that this is something that might be considered for a future version. The same effect can be achieved currently as follows:

```
std::tuple f()  
[[ensures r: get<0>(r)>0 && get<1>(r).size()>0]];
```

2.5 Information of contract_violation

The information in **contract_violation** could be partially represented by a **source_location** object, from the Library Fundamentals V2 Technical Specification.

However, we defer this decision for a future version of this proposal.

2.6 Name lookup of contracts

Name lookup resolution in contracts may interact with the use of different build modes.

There are two answers here. The first one would be to make resolution dependent on the current build mode, requiring that only the contracts that would be evaluated in the current build mode need to parse correctly and pass the name lookup.

A second solution would be to make name lookup independent of build mode. In that case, all contracts would be required to pass name lookup independently of their assertion level. We have selected this second solution.

2.7 Identical contracts

We are also requiring in the wording that a redeclaration of a function that contains a contract needs to use the same contract (in the sense of ODR) that was present in the first declaration of the function.

The exact meaning would be to require contracts to be lexically the same token by token. This is a simpler definition, but would require that a redeclaration uses also the same names for functions parameters (if/when they are used in the contract).

A second solution would be to require the contracts to be logically equivalent which seems to introduce a number of additional implementation challenges.

A third option is to say that the functions must be textually equivalent except for a change of (parameter) variable names, but otherwise having the same structure. We have selected this option.

We have taken the route of requiring contract expressions to be odr-identical, except for the change of function parameter names.

2.8 Throwing violation handler

If a violation handler throws an exception, it is necessary to clarify what is the effect when the continuation mode is *off*.

One option could be that the exception propagates as the handler did not *return*. However, that design option would open the opportunity for continuation when the continuation has been set to *off*.

Another design alternative would be to unconditionally invoke **terminate()**.

2.9 Additional information in contract violation

Besides original information in **contract_violation**, a new member has been added to store the **assertion_level**. This value contains a string with the *assertion-level* of the contract that has been violated.

2.10 Invoking the handler

The proposal does not support the direct invocation of the violation handler. Allowing so, would imply access to handler supplied by the user, and could result into a security issue. Instead, we have added an additional *assertion-level* named **always**. Such assertion cannot be disabled and the check is performed even when the program is built with *build-level* set to *off*.

```
void f() {
  int x=get_value();
  [[assert always: x>0]]; // Invoke handler if x non-positive
  // ...
  if (x!=0) [[assert always: false]]; // Invoke handler if x!=0
  // ...
  [[assert always:false]]; // Unconditionally invoke handler
  // ...
}
```

This assertion level is available only for `[[assert]]`. A future extension might consider the usefulness of **always** for preconditions and postconditions.

Background of always level

The addition of the *always* assertion level was introduced during the Kona meeting. We reproduce here the relevant results from straw polls in the Evolution Working Group.

Question	SF	F	N	A	SA
Accept and proceed to LEWG	13	6	5	4	4
Accept and proceed to LEWG without <i>always</i>	5	13	9	5	2

Also, an up-down vote was taken:

Proposal as is	Proposal without <i>always</i>
14	9

2.11 Initialization of contract_violation objects

When a contract is broken, a **contract_violation** needs to be created with the corresponding information. There are several options on how such object should be populated with information.

Among the available options, one could be to leave this details as something to be defined by implementations. Otherwise, the exact population approach would need to be standardized.

This proposal establishes some constraints on the values that a contract violation object should hold.

- **Information on source location:** In the case of a precondition violation the source location will be the one from the caller site. In the case of a precondition violation the source location violation will be the one from the callee site. In the case of an assertion, the source location will be the one from the assertion itself.
- **Comment:** It will contain at least the text of the *conditional-expression* that was not satisfied.
- **Assertion level:** It will contain a string representation of the *assertion-level* of the contract that has failed.

3. Questions about contracts programming

What is the effect of violating a contract while evaluating the check for another expression

Nothing special needs to be considered. When the first contract is violated the corresponding action is taken.

```
bool positive(const int * p) [[expects: p!=nullptr]] {
  return *p > 0;
}

bool g(int * p) [[expects: positive(p)]];

void test() {
  g(nullptr); // Contract violation when calling positive(nullptr)
}
```

Contract attribute as identifier

What happens if you try to use an identifier named as a contract attribute (e.g. **requires**, **audit**, ...)? Example:

```
x f(x & audit) [[ensures audit: audit.valid()]];
```

This is valid code. The first `audit` is interpreted as a *contract-level*. Then, the *conditional-expression* identifies the second `audit` correctly as the function argument.

4. Proposed Wording

In 6.2/6 [basic.def.odr], add a new bullet after bullet 6.5:

- if `D` contains an assertion, has a postcondition, or invokes a function with a precondition, each definition of `D` shall be translated using the same build mode (10.6.11 [dcl.attr.contracts]); and

In 10.6.1/1 [dcl.attr.grammar], modify the production for *attribute* as follows:

```
...
attribute-specifier:
    [[attribute-using-prefixopt attribute-list]]
    contract-attribute-specifier
    alignment-specifier
...
```

Add a new section 10.6.11 Contracts attributes [dcl.attr.contracts]

1. Contract attributes are used to specify to preconditions, postconditions, and assertions for functions.

contract-attribute-specifier:

```
[[ expects contract-levelopt : conditional-expression ]]
[[ ensures contract-levelopt identifieropt : conditional-expression ]]
[[ assert contract-levelopt : conditional-expression ]]
```

contract-level:

```
always
default
audit
axiom
```

2. A *contract-attribute-specifier* using `expects` is a *precondition*. It expresses a function's expectation on its arguments and/or the state of other objects using a predicate that should hold upon entry into the function.

3. A *contract-attribute-specifier* using `ensures` is a *postcondition*. It expresses a condition that a function should ensure for the return value and/or the state of objects using a predicate that should hold upon exit from the function.

4. A *contract-attribute-specifier* using `assert` is an *assertion*. It expresses a condition that should be satisfied where it appears in a function body.

5. A precondition or postcondition may be applied to the function type of a function declaration. The first declaration of a function shall specify all preconditions and postconditions (if any) of the function. Subsequent declarations shall either specify no preconditions and postconditions or the same list of preconditions and postconditions. Two lists of preconditions and postconditions are the same if they consist of the same preconditions and postconditions in the same order. Two preconditions or postconditions are the same if their contract levels are the same and their *conditional-expressions* are the same. Two *conditional-expressions* contained in *contract-attribute-specifiers* are the same if they would satisfy the one-definition rule (6.2 [basic.def.odr]) if they appeared in function definitions, except for renaming of parameters and return value identifiers (if any).

6. An assertion may be applied to a null statement (9.2 [stmt.expr]). The *conditional-expression* of an assertion may odr-use any object that can be accessed from the statement it is applied to.

7. Preconditions, postconditions, and assertions are collectively called *contracts*. A contract shall have no observable effect in a program where all contracts would be satisfied if they were evaluated.

8. The *conditional-expression* of a *contract-attribute-specifier* is a potentially-evaluated expression (6.2 [basic.def.odr]). If evaluating the *conditional-expression* would modify a non-local object, the behavior is undefined. If the *conditional-expression* of a *contract-attribute-specifier* appearing in a `constexpr` function odr-uses a non-local object that is not `constexpr`, the program is ill-formed. [Example

```
void push(int x, queue & q)
    [[expects: !q.full()]]
    [[ensures: !q.empty()]]
{
    //...
    [[assert: q.is_valid()]];
    //...
}

int min=-42;
constexpr int max=42;

constexpr int g(int x)
    [[expects: min<=x]] // error
    [[expects: x<max]] // OK
{
    //...
    [[assert: 2*x < max]];
    [[assert: ++min > 0]]; // undefined
    //...
}
```

— end example]

9. If the *contract-level* of a *contract-attribute-specifier* is absent, it is assumed to be `default`. [Note: An *always contract-level* is expected to be used for those contracts where the cost of run-time checking is assumed to be very small compared to the cost of executing the function and that should be checked regardless of the build mode. A *default contract-level* is expected to be used for those contracts where the cost of run-time checking is assumed to be small (or at least not expensive) compared to the cost of executing the function. An *audit contract-level* is expected to be used for those contracts where the cost of run-time checking is assumed to be large (or at least significant) compared to the cost of executing the function. An *axiom contract-level* is expected to be used for those contracts that are formal comments and are not evaluated at run-time. — end note]

10. A translation may be performed with one of the following *build levels*: *off*, *default*, or *audit*. A translation with build level set to *off* performs checking for `always` contracts. A translation with build level set to *default* performs checking for `always` and `default` contracts. A translation with build level set to *audit* performs checking for `always`, `default`, and `audit` contracts. [Note: The mechanism for selecting the build level is implementation-defined. If no build level is selected, the build level is *default*. — end note] There shall be no programmatic way of setting, modifying or querying the build level of a translation unit.

11. If a function has multiple preconditions or postconditions that would be checked, their evaluation will be performed in the order they appear lexically.

```
void f(int * p)
    [[expects: p!=nullptr]]
    [[expects: *p == 0]] // Evaluated after p!=nullptr
{
    *p = 1;
}
```

12. A *violation handler* function is a function that is invoked when a contract violation is detected and that has the following signature:

```
void(const std::contract_violation &);
```

There shall not be any programmatic way of setting or modifying which violation handler is called in the event of a contract violation. It shall be implementation defined how the violation handler is established for a program and how the `std::contract_violation` argument value is set. If a precondition is violated, the source location of the violation shall be the source location of the invocation to the function. If a postcondition is violated, the source location of the violation shall be the source location of the function definition. If an assertion is violated, the source location of the violation shall be the source location of statement to which the assertion is applied to.

13. If a user-provided violation handler exits by throwing an exception and a contract is violated in a call to a function with a *non-throwing* exception specification, then the behavior is as if the exception was thrown within the function body. [Note: The function `std::terminate()` will be invoked. No stack unwinding will be performed. — *end note*] [Example:

```
void f(int x) noexcept [[expects: x>0]];

void g() {
    f(0); // std::terminate() if violation handler throws
    //...
}
```

— *end example*]

14. A translation may be performed with a *violation continuation mode*, which can be: *off* or *on*. A translation with a violation continuation mode set to *off* shall terminate execution by invoking `std::terminate()` after completing the execution of the violation handler. A translation with a violation continuation mode set to *on* shall continue execution after completing the execution of the violation handler. [Note: If no *continuation mode* is selected, the default *continuation mode* is *off*. — *end note*] [Note: A *continuation mode* set to *on* provides the opportunity to install a logging handler to instrument a pre-existing code base and fix errors before enforcing checks. — *end note*] [Example:

```
void f(int x) [[expects: x > 0]];

void g() {
    f(0); // std::terminate() after handler if continuation mode is off
        // Proceeds after handler if continuation mode is on
    //...
}
```

— *end example*]

Add a new section 10.6.11.1 Interface contracts [dcl.attr.contracts.interface]

1. The *contract-attribute-specifiers* `expects` and `ensures` may be only applied to function types. They may appear multiple times applied to a function type with the same or different *contract-levels*. [Example:

```
int z;

bool is_prime(int k);

void f(int x)
    [[expects: x>0]]
    [[expects audit: is_prime(k)]]
    [[ensures: z>10]]
{
    //...
}
```

— *end example*]

2. A postcondition may introduce an identifier to represent the return value of the function. [Example:

```
int f(char * c)
    [[ensures res: res>0 && c!=nullptr]];

int g(double * p)
    [[ensures audit res: res!=0 && p!=nullptr && *p<=0.0]];

— end example]
```

3. If a postcondition odr-uses in its *conditional-expression* a parameter value and the function body modifies that value the program is ill-formed. [Example:

```
int f(int x)
    [[ensures r: r==x]]
{
```

```

    return ++x; // Ill-formed: x modified
}

int g(int * p)
    [[ensures r: p!=nullptr]]
{
    *p = 42; // OK. p is not modified
}

```

— end example]

4. The *conditional-expression* of a precondition or a postcondition of a function may odr-use the function's arguments. If the function is a function template or a member function of a class template, the *conditional-expression* may odr-use the template arguments.

5. The *conditional-expression* of a member function shall not use members that cannot be accessed by a caller to that member function. [Example:

```

class X {
public:
    int v() const;
    void f() [[expects: x>0]]; // Ill-formed
    void g() [[expects: v()>0]]; // OK
protected:
    int w();
    void h() [[expects: x>0]]; // Ill-formed
    void i() [[ensures: y>0]]; // OK
    void j() [[ensures: w()>0]]; // OK
    int y;
private:
    void k() [[expects: x>0]]; // OK
    int x;
};

class Y : public X {
public:
    void a() [[expects: v()>0]]; // OK
    void b() [[ensures: w()>0]]; // Ill-formed
    void c() [[expects: x>0]]; // Ill formed
protected:
    void d() [[expects: w()>0]]; // OK
    void e() [[ensures: x>0]]; // Ill-formed
};

```

— end example]

6. A function pointer shall not include preconditions or postconditions. A call through a function pointer to functions with preconditions or postconditions shall perform contract assertions checking once. [Example:

```

typedef int (*fpt)() [[ensures r: r!=0]]; // Ill-formed

int g(int x)
    [[expects: x>=0]]
    [[ensures r: r>x]]
{
    return x+1;
}

int (*pf)(int) = g; // OK

```

— end example]

Add at the end of clause 13.3 [class.virtual]:

18. An overriding function shall have the same list of preconditions and postconditions as the overridden function. The list of preconditions and postconditions in the overriding function may be omitted, in which case it is implicitly assumed to be the list of the overridden function; if there is more than one overridden function all such lists shall be the same.

Modify clause 20.5.1.3/2 [compliance]:

2. A freestanding implementation has an implementation-defined set of headers. This set shall include at least the headers shown in Table 19.

Table 19 — C++ headers for freestanding implementations

	Subclause	Header(s)
		<ciso646>
21.2	Types	<cstdint>
21.3	Implementation properties	<float> <limits> <climits>
21.4	Integer types	<cstdint>
21.5	Start and termination	<cstdlib>
21.6	Dynamic memory management	<new>
21.7	Type identification	<typeinfo>
21.8	Contract violation handling	<contract>
21.89	Exception handling	<exception>
21.910	Initializer lists	<initializer_list>
21.1011	Other runtime support	<cstdint> <cstdlib> <stdbool>
23.15	Type traits	<type_traits>
Clause 32	Atomics	<atomic>
D.4.2, D.4.3	Deprecated headers	<cstdint> <stdbool>

Modify clause 21.1/2 [support.general]:

2. The following subclauses describe common type definitions used throughout the library, characteristics of the predefined types, functions supporting start and termination of a C++ program, support for dynamic memory management, support for dynamic type identification, support for contract violation handling, support for exception processing, support for initializer lists, and other runtime support, as summarized in Table 32.

Table 32 — Language support library summary

	Subclause	Header(s)
21.2	Common definitions	<cstdint> <cstdlib>
21.3	Implementation properties	<limits> <climits> <float>
21.4	Integer types	<cstdint>
21.5	Start and termination	<cstdlib>
21.6	Dynamic memory management	<new>
21.7	Type identification	<typeinfo>
21.8	Contract violation handling	<contract>
21.89	Exception handling	<exception>
21.910	Initializer lists	<initializer_list>
21.1011	Other runtime support	<signal> <setjmp> <cstdint> <cstdlib> <stdbool> <cstdlib>

21.9 Contract violation handling

1. The header `<contract>` defines a type for reporting information about contract violations generated by the implementation.

21.9.1 Header `<contract>` synopsis

```
namespace std {  
    class contract_violation;  
}
```

21.9.2 Class `contract_violation`

```
namespace std {  
    class contract_violation {  
    public:  
        int line_number() const noexcept;  
        string_view file_name() const noexcept;  
        string_view function_name() const noexcept;  
        string_view comment() const noexcept;  
        string_view assertion_level() const noexcept;  
    };  
}
```

1. The class `contract_violation` describes information about a contract violation generated by the implementation.

```
int line_number() const noexcept;
```

2. *Returns:* The source code line number where the contract violation happened (10.6.11/12 [`decl.attr.contracts`]).

3. *Remarks:* An implementation may return `-1` if the location is unknown.

```
string_view file_name() const noexcept;
```

4. *Returns:* The source file name where the contract violation happened (10.6.11/12 [`decl.attr.contracts`]).

5. *Remarks:* An implementation may return `nullptr` if the location is unknown.

```
string_view function_name() const noexcept;
```

6. *Returns:* The name of the function where the contract violation happened (10.6.11/12 [`decl.attr.contracts`]).

7. *Remarks:* An implementation may return `nullptr` if the function is unknown.

```
string_view comment() const noexcept;
```

8. *Returns:* The text of the contract that was violated.

9. *Remarks:* The text will include a textual representation from the *conditional-expression* that was not satisfied.

```
string_view assertion_level() const noexcept;
```

10. *Returns:* The text with the *assertion-level* of the contract that was violated.

Acknowledgements

Thanks to Jens Maurer for his great help in the detailed wording.

Alexander Beels, Andrzej Krzemienski and Melissa Mears reviewed previous versions of this document.

References

- [1] G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup. A contract design. [P0380R1](#). July, 2016.
- [2] Richard Smith. Working Draft, Standard for Programming Language C++. [N4700](#) October, 2017.